Department of Computer Science & Engineering

Operating Systems

UE24CS242B

OS Unit 4 Orange Problem

GITHUB LINK: https://github.com/Suryapurimetla/PES1UG24AM210-pes-vcs

Name : PURIMETLA JAYA SURYA
SRN : PES1UG24AM210
Section: D
Environment: Ubuntu 22.04 / local repository completion

Implemented Files

- object.c: object_write and object_read implemented with hashing, sharding, integrity checking, and atomic writes.
- tree.c: tree_from_index implemented with recursive subtree construction and tree object writing.
- index.c: index_load, index_save, and index_add implemented for the text-based staging area.
- commit.c: commit_create implemented and commit parsing fixed for correct log output.
- report.md and screenshots/: updated with screenshot references and written answers.

Implementation Summary

- Phase 1 implements content-addressable object storage with hash verification. Phase 2 implements tree serialization and root tree construction from staged files. Phase 3 implements the text-based staging index with atomic saves. Phase 4 implements commit creation, reference updates, and log traversal.

Validation Summary

- ./test_objects passes for object storage behavior. ./test_tree passes for tree parsing and deterministic serialization. Manual pes init, pes add, pes status, pes commit, and pes log flows work. make test-integration completes successfully.
-

Screenshots

Phase 1

Screenshot 1A — ./test_objects output

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

This confirms successful blob storage, deduplication, and integrity checking in the object store implementation.

Screenshot 1B — find .pes/objects -type f

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

The object files are sharded by the first two hexadecimal characters of the hash, which matches the intended content-addressable storage layout.

Phase 2

Screenshot 2A — ./test_tree output

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

This shows that tree serialization and parsing are consistent and that the serialized ordering is deterministic.

Screenshot 2B — raw tree object via xxd

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e1
1ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
```

The raw dump shows the tree header followed by serialized entries containing mode, filename, and binary object hashes.

Phase 3

Screenshot 3A — pes init -> pes add -> pes status

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ ./pes init
echo hello > file1.txt
echo world > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  pes.c
  untracked:  test_sequence.sh
  untracked:  tree.h
  untracked:  index.h
  untracked:  file.txt
  untracked:  index.c
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  pes.h
  untracked:  Makefile
  untracked:  commit.c
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  README.md
  untracked:  test_objects.c
```

The status output shows that staged files are tracked correctly and that no unstaged changes are reported immediately after adding them.

Screenshot 3B — cat .pes/index

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776760214 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776760214 6 file2.txt
```

The index stores the file mode, blob hash, modification time, size, and path in the expected text format.

Phase 4

Screenshot 4A — pes log

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ echo World >> file1.txt
./pes add file1.txt
./pes commit -m "Add world"

echo Goodbye > file3.txt
./pes add file3.txt
./pes commit -m "Add farewell"

./pes log
Committed: d9f226366b00... Add world
Committed: a716083f2844... Add farewell
commit a716083f28441bf402b6f6f9fa365a964d860f1f7d40ae25f665095f8036069f
Author: PES User <pes@localhost>
Date:   1776760711

    Add farewell

commit d9f226366b0029bcaca123f104b6685d594fdfd298f8e6dd38d62fa78ef0be33
Author: PES User <pes@localhost>
Date:   1776760711

    Add worldell

commit e096939aa65f0a13ae386c068bb19f0fb1fce25b4b80ae419e1ae09998adad1d
Author: PES User <pes@localhost>
Date:   1776760532

    Initial committial commiQ
```

The log confirms that commits are linked through parents and printed from newest to oldest with author, timestamp, and message metadata.

Screenshot 4B — find .pes -type f | sort

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0a/74dd15b30360b901616d836e751f9984c526cd9bfb6775c79c53c6c19d8bc9
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/d488e897a9291aca9f7f55db526cf1c0a4d2a6274de8d6729d8da036cce9fd
.pes/objects/15/69038b7a09b20bfeb5b713e1231d45db5182501507706c7afd82f0724f94cd
.pes/objects/2c/f773d0de14bcf8e2ae5b3511b22f25f8238d8308aa0e145e53d98d9ab32839
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/34/3d34bf690fefa64dc0d95a01280df0e5d489c8eced213340be03ea5ffacc2b
.pes/objects/36/c631b95e4c99b681fce6fa71fa9008a9ecd57f8965afe34f4dca50adeb2171
.pes/objects/3d/d9f832ab4ce50c29641e8fdb7bc1c4f5a4710008008ce332e25857671d85ac
.pes/objects/5e/88dc2d6f8241ffda1a0d3f07cbf02b76ecf0726c10377b36f04ee35848a362
.pes/objects/60/459622a6a99c40ddfcd668e7cbd0d70a4d53aeb448a251aa982ea2043e3838
.pes/objects/6b/1a89bbbd6a604d675a2fa7e4d59aa0d38e3867805722b60e37aa21bb195414
.pes/objects/6e/2ece108ec707b076dc85e5691d8d82601dc9d5e4b275d26d30eadb8ccdda81
.pes/objects/7c/decd5b5adfe3db82fd782549031cd9b3e23b6c38ef7a97293330e9a4ec2194
.pes/objects/7c/fec9415148b36d69ba32f573d331ff32307f40b106232dcc6e9e37405219cb
.pes/objects/86/ea2a41822f213daa421afaaec58eb0160b8a08c3376085fee932ef1af6f043
.pes/objects/8e/128fab627eae587dffd5b1bf4bc8e5a7a75af7c3aac302fc408cb35f022b5a
.pes/objects/a7/4c8d33a8cdae4949c89c9996cf1d246bbd04c40826d9dc7d25fbe611236b73
.pes/objects/a9/ca281f9faa54152e0aadb9d5f4713f4e056c36307dd02aa41b36f4f9a464a9
.pes/objects/af/33093be69271a22dc4344ce07b65ad21abfb5de7f330d0d960d8824212a993
.pes/objects/c9/ed899783ff70cac61dbbc4e205d04714402c853c4e5f6107d2c8b21cd3a8f4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e0/96939aa65f0a13ae386c068bb19f0fb1fce25b4b80ae419e1ae09998adad1d
.pes/objects/e9/5829cbebfc4ff105b206ece482623e221adfa1973ba6a858531b6d74739708
.pes/objects/f7/7c03788f0e18ff8b43c8134f449b8f19b890cc74faf3329e4c9bb978aefc03
.pes/objects/f7/c861f242399b9f695a19c977edddef2185bb6e0f3fe2b2eda2e0520fe2fa44
.pes/refs/heads/main
```

This listing shows the growth of repository metadata after multiple commits, including refs, the index, and newly created objects.

Screenshot 4C — cat .pes/refs/heads/main and cat .pes/HEAD

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
e096939aa65f0a13ae386c068bb19f0fb1fce25b4b80ae419e1ae09998adad1d
ref: refs/heads/main
```

The output demonstrates the expected reference chain: HEAD points to the branch name, and the branch file points to the latest commit hash.

Final Integration Test

Integration Screenshot — make test-integration

```
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: c9cfe60b9c4d... Initial commit

Log after first commit:
commit c9cfe60b9c4dae71350e27b69c3d7907ec1d777a9c632f3f302e9e8bf1e3ff1d
Author: PES User <pes@localhost>
Date:   1776760918

    Initial commit


--- Second Commit ---
Committed: 4a4d0f86f1f4... Update file.txt

--- Third Commit ---
Committed: 3746ca2c8a94... Add farewell

--- Full History ---
commit 3746ca2c8a94761dd59c6e05861e6aacce3effa1782ab766efe2665de8c4506f
Author: PES User <pes@localhost>
Date:   1776760918

    Add farewell

commit 4a4d0f86f1f4a5baa66befd1899103712632eb28de9eba053c723922f886672b
Author: PES User <pes@localhost>
Date:   1776760918

    Update file.txt

commit c9cfe60b9c4dae71350e27b69c3d7907ec1d777a9c632f3f302e9e8bf1e3ff1d
Author: PES User <pes@localhost>
Date:   1776760918

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
3746ca2c8a94761dd59c6e05861e6aacce3effa1782ab766efe2665de8c4506f

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/37/46ca2c8a94761dd59c6e05861e6aacce3effa1782ab766efe2665de8c4506f
.pes/objects/4a/4d0f86f1f4a5baa66befd1899103712632eb28de9eba053c723922f886672b
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c9/cfe60b9c4dae71350e27b69c3d7907ec1d777a9c632f3f302e9e8bf1e3ff1d
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
surya_purimetla@LAPTOP-ENQCQDS7:~/PES1UG24AM210-pes-vcs$
```

Together, these screenshots verify the full end-to-end workflow from repository initialization through staging, committing, history traversal, and reference updates.

Analysis Answers

Q5.1 Branching and Checkout

A branch in PES-VCS should be implemented as a file under .pes/refs/heads/<branch> that stores the target commit hash, exactly like the existing main branch file. A pes checkout <branch> operation would first verify

that the branch file exists, then rewrite .pes/HEAD so that it contains ref: refs/heads/<branch>. After that, the working directory must be updated to match the tree pointed to by the branch's latest commit. That means reading the target commit object, reading its root tree, recursively materializing every file and directory into the working directory, and removing tracked files that exist in the current branch but not in the target branch. The complexity comes from the fact that checkout is not only a metadata change in .pes/; it also has to safely transform the user's actual filesystem state without overwriting uncommitted work or leaving the repository half-updated if an error occurs in the middle.

Q5.2 Dirty Working Directory Conflict Detection

The conflict check can be done by comparing three states: the HEAD snapshot, the index, and the working directory. For each tracked path in the current branch, PES-VCS should determine whether the working directory still matches the staged version recorded in the index. The fast path is to compare stat() metadata such as modification time and size against the index entry. If those values no longer match, the safe path is to re-read the file, hash it, and compare that hash with the blob hash stored in the index or current HEAD tree. If a file has local modifications and the target branch also wants a different version of that same path, checkout must refuse because switching branches would overwrite user data. In short, PES-VCS should reject checkout when a tracked file is dirty in the working tree and the target branch would update or delete that path.

Q5.3 Detached HEAD

In detached HEAD state, .pes/HEAD would contain a commit hash directly instead of a symbolic reference like ref: refs/heads/main. New commits made in this state still work: each new commit records the previous detached commit as its parent, so history continues normally, but no branch name moves forward to remember those commits. As a result, the commits become hard to find later unless the user notes the hash manually. Recovery is straightforward if the user still knows one of the detached commit hashes: create a new branch file under .pes/refs/heads/ pointing to that commit, or update an existing branch to that hash. Once a reference points to the detached history again, the commits are no longer orphaned and can be reached through normal log traversal.
Design assumption for Q5: these checkout answers assume PES-VCS updates only tracked files and rebuilds the working directory from tree objects, rather than trying to preserve arbitrary untracked files automatically.

Q6.1 Garbage Collection and Space Reclamation

Garbage collection should use a mark-and-sweep algorithm. The mark phase starts from every live reference, such as all files under .pes/refs/heads/ and also HEAD if it stores a direct commit hash. For each referenced commit, PES-VCS recursively marks the commit itself, its parent commits, its root tree, every subtree reachable from that tree, and every blob reachable from those trees. A hash set is the right data structure for tracking reachability efficiently, because membership tests and inserts are close to O(1). After marking, the sweep phase walks .pes/objects/ and deletes any object whose hash is not in the reachable set. For a repository with 100,000 commits and 50 branches, the number of visited objects is on the order of the distinct reachable commits, trees, and blobs, not 100,000 * 50, because many branches usually share large parts of history. In practice, GC would still need to visit a very large number of objects, likely hundreds of

thousands or more in a moderately sized repo.

Q6.2 Why Concurrent GC Is Dangerous

Running garbage collection concurrently with commit creation creates a race between "object becomes reachable" and "reference is updated." During commit, PES-VCS first writes blobs, trees, and the new commit object, and only at the end updates the branch reference. If GC scans references in the middle of that sequence, it may not yet see the new commit from any branch, so it can conclude that the newly written tree or commit objects are unreachable and delete them. Immediately after that, the commit path might update refs/heads/main to point at a commit whose underlying objects were just removed, corrupting the repository. Real Git avoids this with coordination and conservative retention: reference updates are atomic, very new objects are protected, and GC is careful not to collect objects that may still become reachable moments later.

Failure consequence for Q6: the most visible symptom would be a branch ref that points to a commit hash which exists, but whose tree or blob dependencies have already been deleted, causing future reads or log/tree walks to fail.